



UADER- FACULTAD DE CIENCIA Y TECNOLOGÍA - SEDE CONCEPCIÓN DEL URUGUAY

CÁTEDRA: Ingenieria de software I.

PROFESOR: Dr. Pedro E. Colla.

AYUDANTE: Lic. Fernando Heit.

ALUMNOS: -Ariadna Rizzo Schauvinhold.

-Eloy Olivera.

-Gisella Soledad Rodriguez.

-Milton Cárdenas.

-Sofia Orellana.

AÑO ACADÉMICO: 2026.

1. Primero, se cambió la línea 59 dentro de **loginClient.html** :

```
<td><input class="txt" name="contacto" type="email" placeholder="Correo electrónico del cliente"><br></td>
```

La cual antes decía `type="txt"` y solo dejaba ingresar número o letras, con éste cambio se pueden ingresar correos. También por la estética se cambió el `placeholder="IDcliente"` por `placeholder="Correo electrónico del cliente"`

Para las funcionalidades, dentro de **loginClient.js** se reemplazaron todas las funciones que pedían como información la id del cliente (.id) por la clase **contacto** que ya estaba definida en el código del profe, ahora JavaScript al momento de analizar y corroborar la información solicita y utiliza el correo (contacto) en vez del id (id)

Debido a un error en el cual al logear el usuario existente la página se quedaba cargando, se implementó una función nueva en la línea 134 de **index.js** para resolverlo:

```
scanDb(contacto)
.then(lista => {
if (lista.length == 0) {
    res.status(400).send({response : "ERROR" , message : "Cliente invalido"});
} else {
    const cliente = lista[0];

    if (password === cliente.password) {
        if (cliente.activo==true) {
            res.status (200).send(JSON.stringify({
                response : "OK",
                id     : cliente.id,
                "nombre" : cliente.nombre,
                "contacto" : cliente.contacto,
                "fecha_ultimo_ingreso" : cliente.fecha_ultimo_ingreso
            }));
        } else {
            res.status(400).send({response : "ERROR" , message : "Cliente inactivo"});
        }

    } else {
        res.status(400).send({response : "ERROR" , message : "usuario incorrecto"});
    }
}
});
```

```
});
```

Busca en la base los clientes cuyo campo contacto sea el correo que llegó. Como la consulta tarda, `.then` espera el resultado y lo guarda en `lista` (la lista de clientes encontrados). Todo lo que sigue corre recién cuando hay respuesta.

Si la lista está vacía, no hay ningún cliente con ese correo. Se responde con `ERROR` y se corta ahí.

`res.status(400)` es el código HTTP de “pedido incorrecto”, y `.send({})` manda el JSON con el mensaje.

Si hay resultados, se toma el primero. `lista[0]` es el cliente encontrado, con todos sus campos: cliente.id, cliente.password, cliente.nombre, etc. Se toma solo el primero porque el correo debería ser único.

```
if (password === cliente.password)
```

Compara la contraseña que mandó el usuario (password, que vino en el cuerpo del pedido) con la guardada en la base (cliente.password). (El === compara valor y tipo)

1. Si la contraseña coincide, se revisa que el cliente esté activo. Si lo está, se responde con código 200 (éxito) y un JSON con los datos que la pantalla necesita: id, nombre, contacto y la fecha de último ingreso.
2. Si la contraseña es correcta pero el cliente está desactivado, se rechaza con ese mensaje.
3. Si la contraseña no coincide, se responde “usuario incorrecto”.

En la línea 118 se borró una función que creía innecesaria:

```
/*---
  /api/loginCliente
  Esta API permite acceder a un cliente por ID y comparar la password
pasada en un JSON en el cuerpo con la indicada en el DB
*/
app.post('/api/loginCliente', (req,res) => {

    const { id } = req.body;
    const {password} = req.body;

    console.log("loginCliente: id("+id+") password ("+password+")");

    if (!password) {
        res.status(400).send({response : "ERROR" , message : "Password
no informada"});
```

```
        return;
    }
    if (!id) {
        res.status(400).send({response : "ERROR" , message : "id no
informado"});
        return;
    }

    let getClienteByKey = function () {
        var params = {
            TableName: "cliente",
            Key: {
                "id" : id
            }
        };
        docClient.get(params, function (err, data) {
            if (err) {
                res.status(400).send(JSON.stringify({response :
"ERROR", message : "DB access error "+err}));
            }
            else {
                if (Object.keys(data).length == 0) {
                    res.status(400).send({response : "ERROR" , message
: "Cliente invalido"});
                } else {
                    const paswd=jsonParser('password',data.Item);
                    const activo=jsonParser('activo',data.Item);
                    const id=jsonParser('id',data.Item);
                    const contacto=jsonParser('contacto',data.Item);
                    if (password == paswd) {
                        if (activo == true) {
                            const
nombre=jsonParser('nombre',data.Item);
                            const
fecha_ultimo_ingreso=jsonParser('fecha_ultimo_ingreso',data.Item);

res.status(200).send(JSON.stringify({response : "OK", "id" : id,
"nombre" : nombre, "contacto" : contacto, "fecha_ultimo_ingreso":
fecha_ultimo_ingreso}));
                        } else {

res.status(400).send(JSON.stringify({response : "ERROR", message :
"Cliente no activo"}));
```

```
                        }
                    } else {
                        res.status(400).send(JSON.stringify({response :
"ERROR" , message : "usuario incorrecto"}));
                    }
            }
            }
        })
    }
    getClienteByKey();

});
```

¿Por qué es innecesaria? Toda ésta función trabaja con la ip, cosa que nosotros ya no solicitamos ni utilizamos.
Usaba `docClient.get`, que solo puede buscar por la clave primaria de la tabla, el id.
Ahora el usuario ingresa con su correo, que no es la clave primaria, sino un campo común.
`get` no puede buscar por un campo común. Hay que usar scan con filtro, que es lo que hace `scanDb()`.

Este código no tiene .catch. Si la consulta a AWS falla (por ejemplo, keys incorrectas o sin conexión), la promesa se rechaza y no hay nadie que responda. El login se queda colgado, y en Node eso puede cerrar el servidor con un error. Es por eso que quedaba cargando y se hizo el arreglo.

2. Dentro de **index.js** se cambió la línea 123:

```
console.log("loginCliente: contacto("+contacto+")");
```

La cual antes aparte de `("+contacto+")` contaba con ("+password+"), y devolvia el correo junto con la contraseña, por eso se quitó y ahora devuelve solo el correo.

Además. en **loginCliente.js** se cambió la línea 141:

```
console.log('users.response=' + users.response);
```

el `users.response` era un `users.password`, con la versión anterior a la consola se mandaba la contraseña del usuario, editado: se manda las respuesta definidas abajo de ésta línea, “OK” o “Inválido”.

```
if (users.response == 'OK') {
```

```
} else {
                    console.log('La password no es correcta');
```

3.

4. Dentro de **listarTicket.js**, línea 34: Se agregó un campo para mostrar el nombre, igual a como se solicitan y muestran los otros datos.

```
"<tr><td>Nombre</td><td>" + query.nombre + "</td></tr>" +
```

5. Se hizo un archivo .html basado en el **loginCliente.html** original del profe para la estética, pero cambiando los campos para que pida type=”email” y type=”password” para evitar que se muestre la contraseña al momento de escribirla.
y un archivo .js para su funcionalidad, obviamente.

La API ahora busca al cliente por correo (antes, el formulario enviaba el correo, pero **/api/resetCliente** esperaba un id; por eso el servidor respondía «Id no informada».) y actualiza su contraseña y la fecha de cambio en DynamoDB. Antes esperaba un ID, por eso aparecía. El cambio está en index.js y **IS1_MesaAyuda.js**. Pide la contraseña nueva dos veces y requiere aceptar los términos en **resetClient.html.**

Flujo del cambio de contraseña:

1. El formulario recoge los datos en **resetClient.html**, los campos tienen estos nombres: contacto para el correo, NewPassword y ConfirmarPassword para las dos contraseñas, y termscondition para la aceptación de los términos.
2. El navegador válida antes de llamar al servidor. En **resetclient.js**:
    - comprueba que correo y contraseñas estén completos;
    - compara las dos contraseñas exactamente;
    - verifica que se hayan aceptado los términos;
3. La API identifica la cuenta por correo. En **index.js** (y en la versión fuente **IS1_MesaAyuda.js**) el endpoint ahora recibe contacto y password, comprueba que estén informados y busca en la tabla cliente el registro cuyo contacto coincide con ese correo.
4. DynamoDB actualiza el registro encontrado. La búsqueda por correo permite recuperar el id, que es la clave utilizada para actualizar la fila en DynamoDB. La operación guarda la nueva contraseña y también la fecha de cambio. Si no encuentra una cuenta, responde con error; si la actualización termina bien, responde response: "OK".
5. El navegador muestra el resultado. Si recibe el OK, muestra la confirmación y un botón para volver al inicio e intentar iniciar sesión con la contraseña nueva. No cambia la base de datos mientras se escribe el formulario: la actualización ocurre al confirmar y completar correctamente la solicitud.

Explicación de registroCliente.js

Ingeniería de Software I | Mesa de Ayuda | Guía para comprender y defender el código

1. ¿Para qué sirve este archivo?
Controla el formulario de registro: recibe los datos de registroCliente.html, los valida y los envía mediante una solicitud HTTP al servidor Node.js. El servidor ejecuta /api/addCliente y gestiona el alta en DynamoDB.

2. Seleccionar el formulario
const formE1 = document.querySelector('.form');

document.querySelector busca el primer elemento HTML con la clase form. Guardamos ese formulario en formE1 para usarlo luego.

3. Interceptar el envío

```
formE1.addEventListener('submit', (event) => {
    event.preventDefault();
```

addEventListener detecta cuando se presiona Registrarse. preventDefault evita el envío automático del navegador: así podemos validar antes de hacer una petición con fetch.

4. Recuperar los datos

```
const formData = new FormData(formE1);
const data = Object.fromEntries(formData);
```

FormData recoge valores mediante los atributos name de cada campo. Object.fromEntries los convierte en un objeto: data.nombre, data.contacto, data.password, data.ConfirmarPassword y, si se marcó la casilla, data.termscondition.

5. Validar campos obligatorios

```
if (data.nombre == '' || data.contacto == '' ||
    data.password == '' || data.ConfirmarPassword == '') {
    document.getElementById('resultado2').style.color = 'RED';
    document.getElementById('resultado2').textContent =
        'Debe completar todos los campos';
    return;
}
```

El operador || significa “o”: basta con que uno de los campos esté vacío para mostrar el error. return detiene esa ejecución y evita enviar datos incompletos. La condición actual no detecta cadenas que contienen solo espacios.

6. Comparar las contraseñas

```
if (data.password != data.ConfirmarPassword) {
    document.getElementById('errorPassword').textContent =
        'Las contraseñas no coinciden';
    return;
}
document.getElementById('errorPassword').textContent = '';
```

!= comprueba si son distintas. Si lo son, se muestra el mensaje en el span errorPassword y se detiene el registro. Si coinciden, se borra ese mensaje. ConfirmarPassword no se envía a Node.js.

7. Aceptar términos y condiciones

```
if (data.termscondition != 'on') {
    document.getElementById('resultado2').style.color = 'RED';
    document.getElementById('resultado2').textContent =
        'Debe aceptar los términos y condiciones';
```

```
    return;
}
document.getElementById('resultado2').textContent = '';
```

Un checkbox marcado suele entregar on a FormData si no tiene value personalizado; si no se marca, normalmente se omite. La condición impide continuar si no hubo aceptación, como solicita el TP.

```
8. Definir la dirección de la API
const RESTAPI = {
    addCliente: 'http://localhost:8080/api/addCliente'
};
```

Live Server muestra el HTML (normalmente puerto 5500). Node.js escucha en 8080 y atiende /api/addCliente. Esta dirección identifica el servicio al que se envía el registro.

```
9. Preparar solamente los datos necesarios
const nuevoCliente = {
    nombre: data.nombre,
    contacto: data.contacto,
    password: data.password
};
```

El objeto tiene los campos que recibe addCliente. No se envían la confirmación de contraseña ni los términos. El ID lo genera el servidor mediante crypto.randomUUID().

```
10. Configurar POST
const options = {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(nuevoCliente)
};
```

POST envía los datos al servidor; Content-Type informa que el cuerpo será JSON; JSON.stringify convierte el objeto JavaScript en una cadena JSON.

```
11. Solicitar el registro con fetch
fetch(RESTAPI.addCliente, options)
    .then((res) => {
        return res.json();
    })
```

fetch hace la petición HTTP. Se utiliza .then porque la operación es asíncrona y devuelve una promesa. res.json interpreta el cuerpo de respuesta como JSON. Una respuesta HTTP 400 no activa por sí sola el catch si el JSON se interpreta correctamente.

```
12. Interpretar la respuesta
.then((cliente) => {
```

```
    if (cliente.response == 'OK') {
        alert('Usuario registrado correctamente');
        window.location.href = 'loginClient.html';
    } else {
        document.getElementById('resultado2').style.color = 'RED';
        document.getElementById('resultado2').textContent =
            cliente.message || 'Error al registrar usuario';
    }
})
```

Si Node.js devuelve response: OK, se muestra una confirmación y se regresa al login. Si responde con otro resultado, se muestra el mensaje devuelto o uno genérico.

13. Manejar fallas con catch

```
.catch((error) => {
    console.error('Error de conexión:', error);
    document.getElementById('resultado2').style.color = 'RED';
    document.getElementById('resultado2').textContent =
        'No se pudo conectar con el servidor';
});
```

catch recoge errores de red y también errores al procesar una respuesta no válida. No prueba necesariamente que el servidor esté apagado.

14. Cómo explicarlo oralmente

Creamos un JavaScript que controla el formulario de registro. Detectamos el envío con addEventListener y usamos preventDefault para validar antes de enviar. Recuperamos los valores con FormData y Object.fromEntries. Verificamos que no falten campos, que ambas contraseñas coincidan y que se acepten los términos. Si todo está correcto, enviamos nombre, correo y contraseña mediante fetch, con método POST y cuerpo JSON, a la API /api/addCliente de Node.js. El servidor comprueba si el cliente existe y realiza el alta en DynamoDB. Si devuelve OK, mostramos el éxito y regresamos al login; en caso contrario, mostramos el error.

15. Diferencia entre frontend y backend

registroCliente.html presenta los campos; registroCliente.js realiza validaciones visuales y solicita el registro; index.js valida y procesa la operación; DynamoDB persiste los datos. El JavaScript del navegador no consulta directamente la base de datos. El código explicado no deshabilita anticipadamente Registrarse si el correo ya existe: ese requisito exige una comprobación adicional. Las validaciones del navegador también deben reforzarse en el servidor.

Nota de seguridad: no se deben registrar ni devolver contraseñas en logs o respuestas. En sistemas reales se almacenan mediante hash seguro y se utiliza HTTPS.

La función `/resumenTickets` que desarrollamos para el proyecto MesaAyuda permite consultar las estadísticas de los tickets de un cliente registrado en el sistema.

Su objetivo es verificar que el cliente tenga permiso para acceder y mostrar la cantidad de tickets que tiene registrados, junto con su información.

¿Cómo funciona paso a paso?

1. Recibe una solicitud GET o POST: utilizamos `app.all()` para permitir ambos métodos. GET recibe los datos mediante `req.query`, mientras que POST los recibe mediante `req.body`.

2. Verifica los datos ingresados: comprueba que se hayan enviado el correo electrónico (`ID`) y la contraseña (`PASSWORD`). Si falta alguno, devuelve un error 400.

3. Busca al cliente: utiliza la función `scanDb(ID)` para consultar la tabla `cliente` de DynamoDB y buscar al usuario mediante su correo electrónico.

4. Valida la contraseña: compara la contraseña ingresada con la almacenada en la base de datos. Si no coincide, devuelve un error 401.

5. Comprueba que el cliente esté activo: verifica que el atributo `activo` sea `true`. Si está inactivo, devuelve un error 403.

6. Busca los tickets: utiliza `scanDbTicket(cliente.id)` para consultar los tickets asociados al identificador interno del cliente.

7. Cuenta los tickets: mediante `resultTickets.length`, obtiene la cantidad de tickets encontrados.

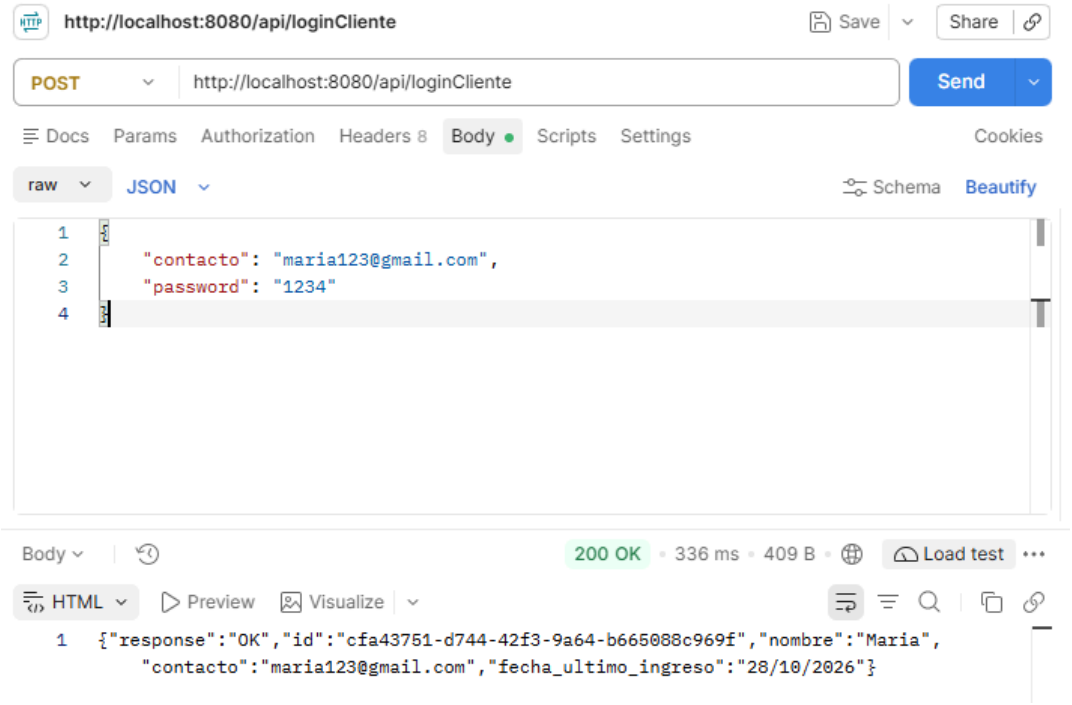
8. Devuelve las estadísticas: si todo es correcto, responde con un JSON que contiene el ID del cliente, su correo electrónico, la cantidad de tickets y el listado de tickets.

9. Controla los errores: utiliza `.catch()` para detectar problemas durante las consultas a DynamoDB y devolver un error 500.

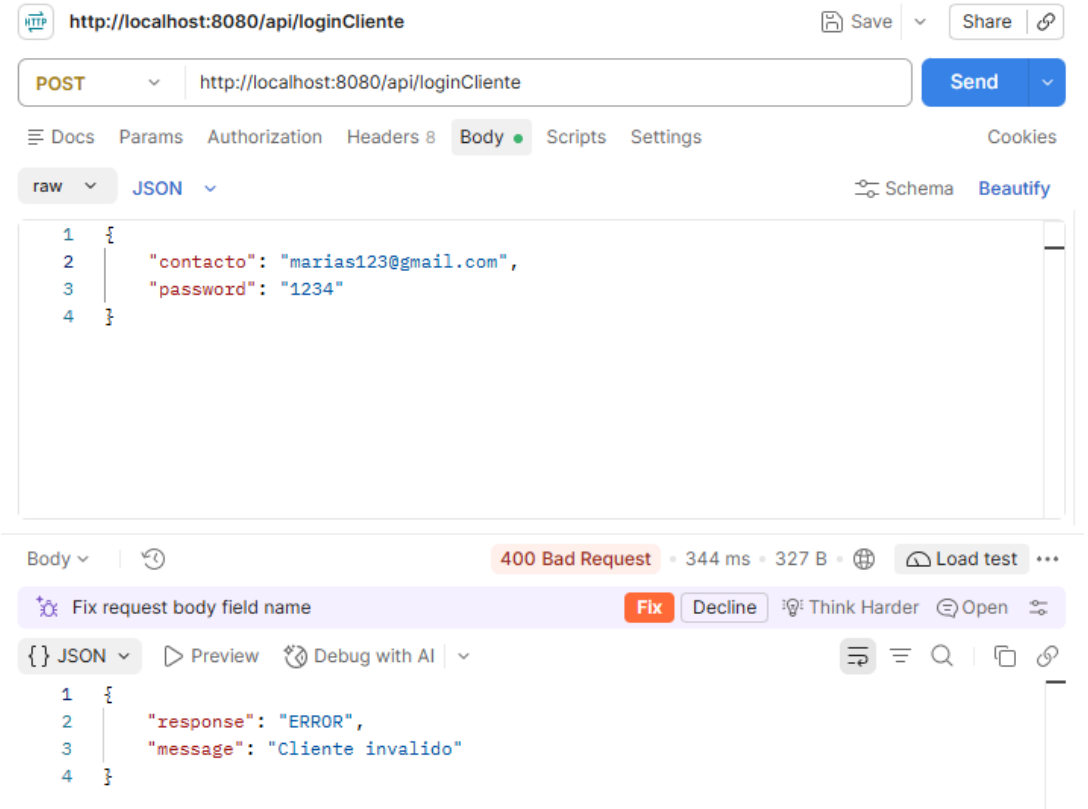
Funciones probadas con Postman

Login de un cliente:

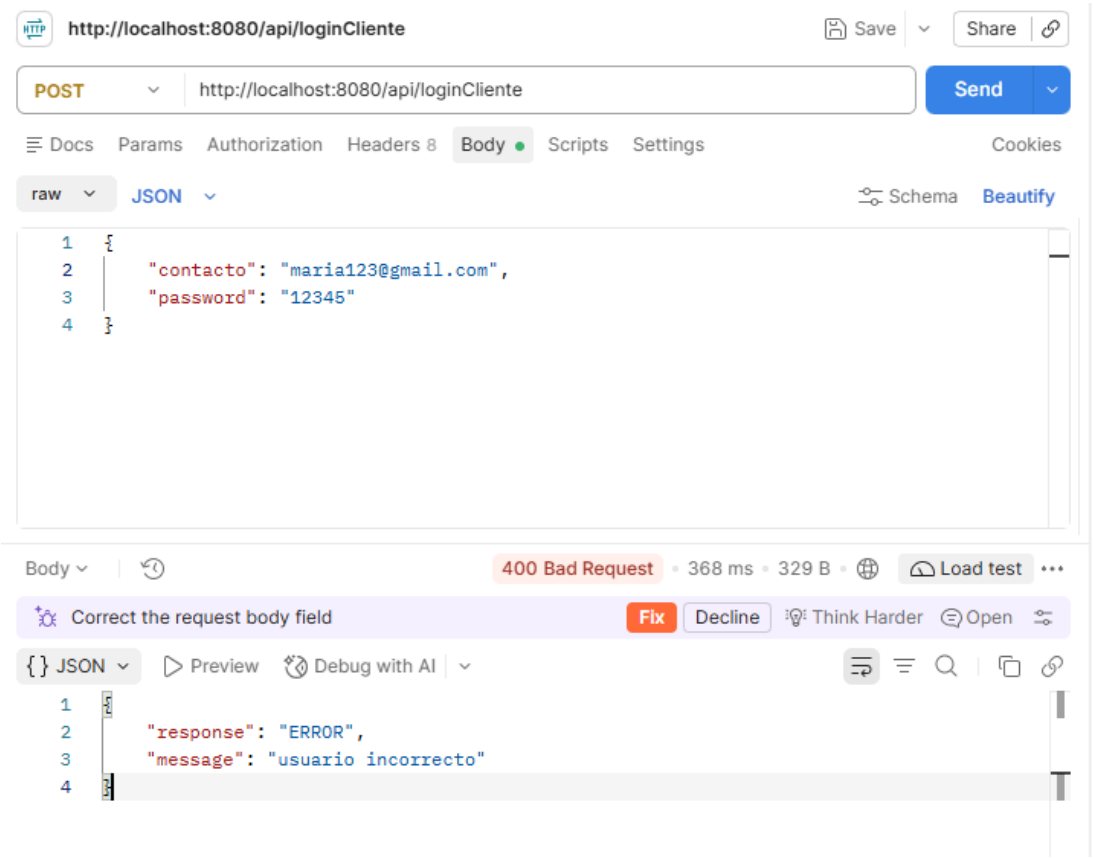
Todo correcto (Camino Feliz)



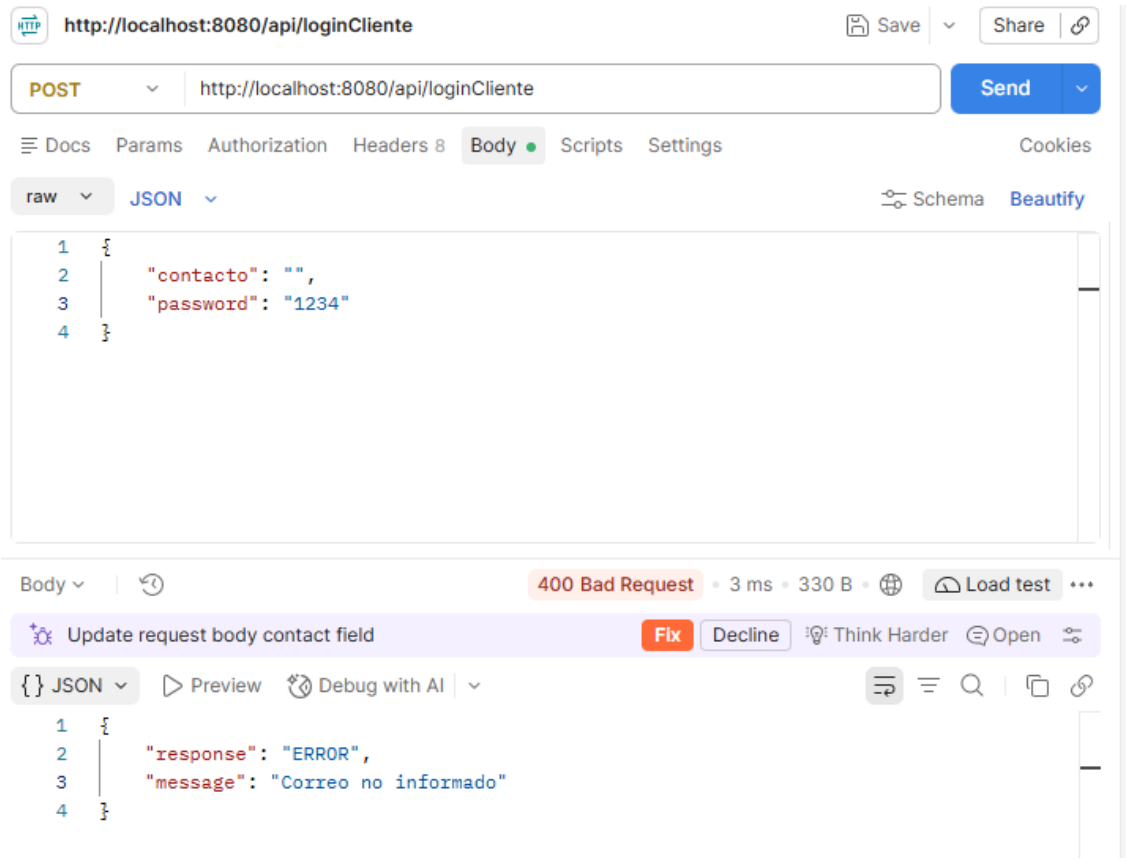
Con correo incorrecto (agregando una s al correo)



Con contraseña incorrecta (agregamos un 5)



Con contraseña pero sin correo



Con correo sin contraseña

